

Notebook

February 8, 2026

1 Wiener's attack

RSA decryption (or signature) creation tends to be relatively energy and time expensive, since we can't just pick a nice private exponent d with low Hamming weight (it would be easily guessable). So one may experience an urge to pick such a public exponent e , that d 's bitlength is much less than the modulus N 's, say if N 's is 2048, than d 's is around 500 bits. Intuition says, that since d is hard to guess (500 bits), you should feel safe anyway. Unfortunately, this is an extremely dangerous mistake to make.

1.1 Theorem (M. Wiener)

Let $N = pq$ with $q < p < 2q$. Let $d < \frac{1}{3}N^{\frac{1}{4}}$. Given (N, e) with $ed = 1 \bmod \varphi(N)$, Marvin can efficiently recover d .

1.1.1 Proof (taken from [Twenty Years of Attacks on the RSA Cryptosystem](#) by Dan Boneh)

The proof is based on approximations using continued fractions. Since $ed = 1 \bmod \varphi(N)$ there exists a k such that $ed - k\varphi(N) = 1$. Therefore,

$$\left| \frac{e}{\varphi(N)} - \frac{k}{d} \right| = \frac{1}{d\varphi(N)}$$

Hence, $\frac{k}{d}$ is an approximation of $\frac{e}{\varphi(N)}$. Although Marvin does not know $\varphi(N)$, he may use N to approximate it. Indeed, since $\varphi(N) = N - p - q + 1$ and $p + q - 1 < 3\sqrt{N}$, we have $\left| N - \varphi(N) \right| < 3\sqrt{N}$

Using N in place of $\varphi(N)$ we obtain:

$$\left| \frac{e}{N} - \frac{k}{d} \right| = \left| \frac{ed - k\varphi(N) - kN + k\varphi(N)}{Nd} \right| = \left| \frac{1 - k(N - \varphi(N))}{Nd} \right| \leq \left| \frac{3k\sqrt{N}}{Nd} \right| = \frac{3k}{d\sqrt{N}}$$

Now, $k\varphi(N) = ed - 1 < ed$. Since $e < N$, we see that $k < d < \frac{1}{3}N^{\frac{1}{4}}$. Hence we obtain:

$$\left| \frac{e}{N} - \frac{k}{d} \right| \leq \frac{1}{dN^{\frac{1}{4}}} < \frac{1}{2d^2}$$

This is a classic approximation relation. The number of fractions $\frac{k}{d}$ with $d < N$ approximating $\frac{e}{N}$ so closely is bounded by $\log_2 N$. In fact, all such fractions are obtained as convergents of the continued fraction expansion of $\frac{e}{N}$. All one has to do is compute the $\log N$ convergents of the continued fraction for $\frac{e}{N}$. One of these will equal $\frac{k}{d}$. Since $ed - k\varphi(N) = 1$, we have $\gcd(k, d) = 1$ and hence $\frac{k}{d}$ is a reduced fraction. This is a linear-time algorithm for recovering the secret key d . **Q.E.D**

1.2 Continued fractions and convergents

Continued fraction is an expression obtained through an iterative process of representing a number as a sum of its integer part and the reciprocal of another number, then writing this other number as the sum of its integer part and another reciprocal and so on. Example: For real $x > 0$ and integers $a_i > 0$, for $i = 1, \dots, n$

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\ddots + \frac{1}{a_n}}}}$$

is a continued fraction. The integers a_0, a_1 , etc., are called the *coefficients* or *terms* of the continued fraction. There are many ways to abbreviate continued fractions. We'll be using this one:

$$x = [a_0; a_1, a_2, \dots]$$

A continued fraction can be approximated by its initial segments. Such approximations are called *convergents*. Obviously, for a rational x the number of such convergents is finite. The first four convergents of a continued fraction are:

$$\frac{a_0}{1}, \frac{a_1 a_0 + 1}{a_1}, \frac{a_2(a_1 a_0 + 1) + a_0}{a_2 a_1 + 1}, \frac{a_3(a_2(a_1 a_0 + 1) + a_0) + (a_1 a_0 + 1)}{a_3(a_2 a_1 + 1) + a_1}$$

If there are successive convergents, they can be calculated recursively. h_i are numerators and k_i are denominators. Then the series can be calculated this way:

$$\frac{h_i}{k_i} = \frac{a_i h_{i-1} + h_{i-2}}{a_i k_{i-1} + k_{i-2}}$$

Now implement the algorithm for finding coefficients and convergents for any given rational number.

Hint: you can use Euclidian algorithm for finding coefficients

[]:

Now that you've implemented the algorithm to find convergents, let's see how to use them to find p and q . Since $\frac{k}{d}$ is a reduced fraction, the denominator of one of the convergents is the d itself. This is enough to decrypt, but we can even factor the N . We also know k , and $\varphi(N) = \frac{ed-1}{k}$, so we can calculate $\varphi(N)$.

$$N - \varphi(N) = pq - (p-1)(q-1) = pq - pq + p + q - 1 = p + q - 1$$

$$q = N - \varphi(N) - p + 1$$

$$N = pq = p(N - \varphi(N) - p + 1) = pN - p\varphi(N) - p^2 + p$$

$$p^2 - p(N - \varphi(N) + 1) + N = 0$$

We have a nice quadratic equation, the roots of which are p and q

Now let's try to apply this knowledge to a vulnerable server. The concept is the same as in the blinding task. You need to provide a valid signature for 'flag' to get the flag. Steps:

1. Find d through the cunning use of continued fractions
2. Find p and q
3. Sign the message and send it to the server
4. Profit

```
[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1338))
        if show:
            print (self.recv_until().decode())
    def recv_until(self, symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_key(self, show=True):
        """Receive public key from the server"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e=int(re.search(r'(?<=e: )\d+', response).group(0))
        N=int(re.search(r'(?<=N: )\d+', response).group(0))
        self.num_len=len(long_to_bytes(N))
        self.e,self.N=e,N
        return (e,N)
    def checkSignatureNumber(self, c, show=True):
        """Check if this number is a valid signature for 'flag'"""
        try:
            num_len=self.num_len
```

```

        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        signature_bytes=long_to_bytes(c,num_len)
        self.checkSignatureBytes(signature_bytes,show)

    def checkSignatureBytes(self,c,show=True):
        """Check if these bytes are a valid signature for 'flag'"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        if len(c)>num_len:
            print ("The message is too long")
            return

        hex_c=c.hex().encode()
        self.s.sendall(b'flag '+hex_c+b'\n',)
        response=self.recv_until(b'\n').decode()

        if show:
            print (response)

        if response.find('Wrong')!=-1:
            print('Wrong signature')
            x=self.recv_until()
            if show:
                print (x)
            return
        flag=re.search(r'CRYPTOTRAINING\{.*\}',response).group(0)
        print ('FLAG: ',flag)
    def setPrivateKey(self,p,q):
        """Set private key"""
        self.p=p
        self.q=q
        self.d=inverse(self.e,(p-1)*(q-1))

    def signMessageBytes(self,m):
        """Sign message after finding private key"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server_
↪first')

```

```

        return
    if len(m)>num_len:
        print('m too long')
    if len(m)<num_len:
        m=bytes([0x0]*(num_len-len(m))) + m
    signature_bytes=long_to_bytes(pow(bytes_to_long(m),self.
↪d,self.N))
    return signature_bytes

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
    (e,N)=vs.get_public_key()

```

```

[ ]:

```

```

[ ]:

```